

ComicCraft: AI Comic Story Creator using Gemini Models

Team ID: UNM123APEXLAB | Date: 29 September 2026

Project Description:

ComicCraft harnesses Generative AI to turn a simple idea into a complete comic story. Users describe their story idea, choose an art style and the number of panels, and the platform generates a panel-by-panel comic script with captions and dialogue, along with an illustration for every panel. The result is displayed in a comic-style layout that can be viewed and downloaded in seconds.

Utilizing Google’s Gemini AI models, ComicCraft uses a Gemini text model to write the structured story script and a Gemini image model to draw each panel. Built with FastAPI and served locally with Uvicorn (and exposed publicly via Ngrok), the platform ensures a fast and user-friendly experience. With secure API key management via .env and responsible input handling, ComicCraft empowers students, writers, creators, and educators to bring their stories to life with confidence.

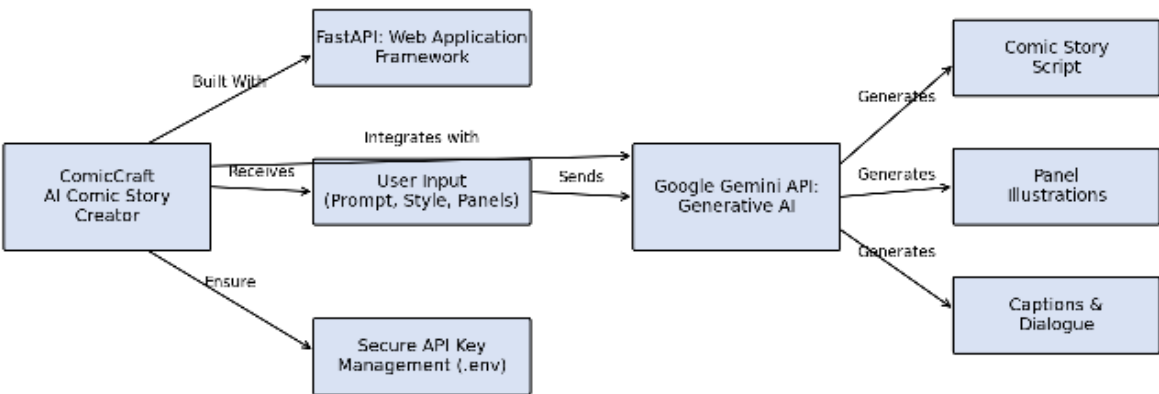
Scenarios

Scenario 1: A user types “A young robot who discovers a garden on an abandoned planet”, selects the Cartoon style and 4 panels. The system generates a four-panel story script with captions and produces a colourful illustration for each panel.

Scenario 2: A teacher wants to explain the water cycle to children. She enters the idea, selects a Manga style, and ComicCraft creates an engaging illustrated comic that turns a difficult topic into a simple visual story.

Scenario 3: A creator brainstorming a superhero origin story selects the Superhero style with 6 panels. The AI produces a dramatic script with dialogue and matching action illustrations, which the creator downloads to use as a storyboard.

Technical Architecture:



Description: ComicCraft utilizes a modular three-tier architecture to deliver rapid, AI-driven comic generation. The frontend provides a responsive user interface built with HTML, CSS, and JavaScript, while the FastAPI-based backend orchestrates data validation and prompt construction. It interfaces with the Google Gemini API, using a text model for story scripting and an image model for panel illustrations, with the API key kept secure in a .env file. The application runs locally

through Uvicorn and is exposed publicly through Ngrok.

Note: ComicCraft does not use a database; all generated content is created on demand and returned directly to the browser, so an ER diagram is not applicable.

Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Gemini API Familiarity:** [Gemini API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Uvicorn Documentation](#)
- **Ngrok for Public Deployment:** [Ngrok Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Gemini models for story generation and image generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI and the Gemini API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Story Script Generator, Panel Illustration Generator, and Style Customization.
- **Activity 2.2:** Implement the FastAPI backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic rendering with JavaScript to display the generated comic based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local FastAPI server over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models from Google Gemini for our comic creation needs. This involves researching the capabilities and performance of the available models, ensuring that the chosen models align with the application's objectives of generating comic scripts, captions, and panel illustrations across different art styles.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with Gemini, you need a valid API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1: Open Google AI Studio.** Visit <https://aistudio.google.com> and sign in with your Google account.

[Screenshot: Google AI Studio home page]

- **Step 2: Navigate to API Keys.** Click "Get API key" from the left-hand menu.
- **Step 3: Create a New API Key.** Click "Create API key", choose a project (or create a new one), and give the key a descriptive name (e.g., comiccraft-key).

[Screenshot: Create API key dialog]

- **Step 4: Copy the Key.** Copy the key immediately and store it in a safe place.
- **Step 5: Add the Key to Your Project.** Create a .env file in the root of your project directory and paste the key as shown below:

```
GEMINI_API_KEY=your_copied_api_key_here
```

- **Step 6: Verify the Key is Loaded.** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
from google import genai

load_dotenv()
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
```

- **Important: Never commit your API key.** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Models

Here are the things that need to be researched to select the best models for the project:

- **Understand the Project Requirements:** Review the specific needs of ComicCraft, focusing on the content it will generate (story scripts, captions, dialogue, and panel illustrations).
- **Explore Gemini's Model Documentation:** Visit the Gemini API documentation to examine the available models, including their capabilities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and suitability for creative storytelling and image generation.
- **Select the Optimal Models:** Choose **gemini-2.5-flash** for structured story script generation because of its balance of quality and speed, and **gemini-2.5-flash-image** for panel illustrations. The story model is used with a temperature of 0.9 and JSON output for rich, varied stories.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML, CSS, JavaScript), the FastAPI backend, and the Gemini API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application by entering a story idea, selecting an art style (Superhero, Manga, or Cartoon), and choosing the number of panels.
- **Outline Backend Responsibilities:** Specify how the FastAPI backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Gemini API.
- **Describe AI Integration Points:** Define how the backend constructs the story prompt, calls the Gemini text model, parses the JSON script, and then calls the Gemini image model for each panel before returning the structured result to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.9+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv
D:\projects>myenv\Scripts\activate
(myenv) D:\projects>pip install -r requirements.txt
```

- **Install FastAPI and Uvicorn:** Install the web framework and the ASGI server:

```
(myenv) D:\projects>pip install fastapi uvicorn jinja2
```

- **Install the Gemini SDK:** Install the Google Gen AI Python library to interact with Gemini:

```
(myenv) D:\projects>pip install google-genai
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects>pip install python-dotenv
```

- **Configure the API Key:** Create a .env file in the project root and add your Gemini API key.
- **Set Up Application Structure:** Create the project directory structure including folders for templates and static files (CSS, JS), and the main application files (app.py, requirements.txt, run_public.py):

```
ComicCraft/
static/
  css/style.css
  js/main.js
templates/index.html
.env
.gitignore
app.py
requirements.txt
run_public.py
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Comic Generation Features

Implement the core generation features as a two-step AI pipeline: a structured story script first, followed by one illustration per panel.

- **Story Script Generator:** Generates a comic title and a panel-by-panel script. Each panel contains a caption or dialogue and a short visual description that is later used as the image prompt.
- **Panel Illustration Generator:** Creates one illustration per panel using the Gemini image model, keeping characters and setting consistent by reusing a shared character and style description in every image prompt.
- **Style Customization:** Lets the user choose an art style (Superhero, Manga, or Cartoon) and the number of panels (3 to 6), which are injected into both the story prompt and the image prompts.

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up routes in `app.py` for the home page and the comic generation endpoint.

Process User Input:

- Create a form in `index.html` for users to submit their story idea, art style, and number of panels.
- Use a Pydantic model to receive and validate the JSON data submitted from the frontend via the Fetch API.
- Validate that `story_prompt` is non-empty and within the 1000-character limit before processing.

Integrate API Calls:

- Within the `/generate` route handler, call `build_story_prompt()` to construct a structured prompt combining the idea, art style, and panel count.
- Submit the prompt to the Gemini text model using `client.models.generate_content()` with JSON output enabled.
- Parse the JSON script, then call the Gemini image model for each panel and return the results to the frontend.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the ComicCraft application. In this milestone, you'll set up the application, establish reliable prompt construction and API interaction, and test each feature to ensure it performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Set Up the Application and Gemini Client:

```
import os, json, base64
from dotenv import load_dotenv
from fastapi import FastAPI, HTTPException, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
from google import genai
from google.genai import types

load_dotenv()
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

TEXT_MODEL = "gemini-2.5-flash"
IMAGE_MODEL = "gemini-2.5-flash-image"

app = FastAPI(title="ComicCraft")
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

class ComicRequest(BaseModel):
    story_prompt: str
    art_style: str = "cartoon"
    panels: int = 4
```

Define the Core Routes in app.py:

Establish routes for ComicCraft's two endpoints, each serving a distinct purpose:

- **/**: Home page route that renders the main index.html interface

```
@app.get("/")
def index(request: Request):
    return templates.TemplateResponse(request, "index.html")
```

- **/generate**: POST endpoint that accepts JSON input, invokes the AI, and returns the structured comic

```
@app.post("/generate")
def generate(req: ComicRequest):
    story_prompt = req.story_prompt.strip()
    style = req.art_style.lower()
    panels = max(3, min(req.panels, 6))

    if not story_prompt:
        raise HTTPException(400, "Story idea is required.")
    if len(story_prompt) > 1000:
        raise HTTPException(400, "Input too long. Please keep it under 1000 characters.")

    try:
        script = generate_script(story_prompt, style, panels)
        comic_panels = []
        for p in script["panels"]:
```

```

        image = generate_panel_image(p["visual"], script["characters"], style)
        comic_panels.append({"caption": p["caption"], "image": image})
    return {"success": True, "title": script["title"],
            "panels": comic_panels, "art_style": style}
except ValueError as e:
    raise HTTPException(500, f"Failed to parse AI response: {e}")
except Exception as e:
    raise HTTPException(500, f"Generation failed: {e}")

```

Set Up Route Handlers for Each Feature:

- For the /generate route, read story_prompt, art_style, and panels from the incoming JSON body.
- Apply input validation: return a 400 error if story_prompt is empty or exceeds 1000 characters.
- Route Fetch requests from the frontend to the /generate endpoint, which processes the data and returns a JSON response.

Define Prompt Engineering Helpers:

- **STYLE_DESCRIPTIONS** dictionary: maps superhero, manga, and cartoon styles to detailed art directions embedded in the image prompts.

```

STYLE_DESCRIPTIONS = {
    "superhero": "bold western comic-book art, dynamic poses, strong outlines, vivid colours",
    "manga": "black-and-white manga style, expressive eyes, screentone shading, dramatic angles",
    "cartoon": "friendly colourful cartoon style, rounded shapes, bright and playful"
}

```

- **build_story_prompt()**: assembles a structured prompt from the story idea, art style, and panel count, instructing the model to return strictly valid JSON.

```

def build_story_prompt(idea: str, style: str, panels: int) -> str:
    return f"""You are an expert comic book writer.
    Write a {panels}-panel comic story for this idea: {idea}
    Art style: {STYLE_DESCRIPTIONS.get(style, 'cartoon')}
    Return ONLY valid JSON in this format:
    {{
      "title": str, "characters": str,
      "panels": [{{
        "caption": str, "visual": str
      }}]
    }}
    """

```

Integrate the Gemini AI Responses in Each Function:

- **generate_script()**: calls the Gemini text model with JSON output enabled, parses the response, and validates that the required keys (title, characters, panels) are present.

```

def generate_script(idea, style, panels):
    response = client.models.generate_content(
        model=TEXT_MODEL,
        contents=build_story_prompt(idea, style, panels),
        config=types.GenerateContentConfig(
            response_mime_type="application/json", temperature=0.9),
    )
    script = json.loads(response.text)
    if not all(k in script for k in ["title", "characters", "panels"]):
        raise ValueError("Invalid response structure from AI")
    return script

```

- **generate_panel_image()**: calls the Gemini image model with the panel's visual description, the shared character description, and the chosen style, and returns the image as a base64 string for the browser.

```

def generate_panel_image(visual, characters, style):
    prompt = (f"Comic panel illustration. {visual}. Characters: {characters}. "
             f"Style: {STYLE_DESCRIPTIONS.get(style, 'cartoon')}. No text in the image.")
    response = client.models.generate_content(model=IMAGE_MODEL, contents=prompt)
    for part in response.candidates[0].content.parts:

```

```
if part.inline_data:
    return base64.b64encode(part.inline_data.data).decode()
raise ValueError("No image returned by the model")
```

- **Home route:** / renders index.html; **/generate (POST)** accepts JSON and returns {success, title, panels, art_style}.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for ComicCraft. This involves designing a responsive layout with HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display the AI-generated comic panels based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the ComicCraft brand and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate icons for visual cues and Google Fonts for clean, comic-style typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a vibrant comic-inspired theme with panel borders and speech-style captions.
- Use flexbox and grid layouts to organize the generated panels into a comic-page grid.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Story Input Form:** a textarea for the story idea, an art style selector (Superhero, Manga, Cartoon), and a panel count selector.
- **Comic Panel Cards:** individual cards per panel, each showing the generated illustration with its caption underneath.
- **Download Button:** lets the user save the finished comic as images.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated loader with a "Creating your comic..." message.

Render Results Dynamically:

- Implement the renderComic(data) function in main.js, which dynamically creates and injects DOM elements for the title and each panel into the results container.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollToView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the ComicCraft application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a new terminal in your code editor and select Command Prompt.
- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
D:\projects>python -m venv myenv
D:\projects>myenv\Scripts\activate
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Gemini API key. The app loads this automatically using python-dotenv at startup:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the FastAPI Development Server:

- Run the application locally using Uvicorn:

```
(myenv) D:\projects>uvicorn app:app --reload
```

- Once running, open a browser and navigate to “http://127.0.0.1:8000” to access the app.
[Screenshot: Terminal showing Uvicorn running on http://127.0.0.1:8000]
- Interact with the comic generator: test all three art styles (Superhero, Manga, Cartoon) and different panel counts to verify that the story scripts and illustrations are correctly generated and rendered.

Activity 5.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local FastAPI server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.

Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the token value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running Uvicorn directly, launch `run_public.py` to start both the Ngrok tunnel and the FastAPI server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 8000 and prints the live public URL in the terminal:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

- Share this URL with anyone who can access your ComicCraft app directly from their browser without any installation.

Important Notes:

- Run the server without the `--reload` option in `run_public.py`, since the reloader would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart `run_public.py` (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart `run_public.py` to get a new URL when this happens.

Exploring the Website's Web Pages:

Home / Generator Page:

[Screenshot: ComicCraft home page]

Description: The single-page interface of ComicCraft serves as both the landing page and the creation tool. It features a vibrant comic-inspired design, a header displaying the ComicCraft brand and a tagline. The main section contains the input form, art style selector, panel count selector, and the AI-generated comic panel all on one seamless page.

Input Section:

[Screenshot: Story input form with a sample idea]

Description: A card containing the main creation form. Users enter their story idea in a textarea (up to 1000 characters), select an art style (Superhero, Manga, or Cartoon), choose the number of panels, and click the Generate Comic button. While the AI processes the request, the button transitions to an animated loading indicator.

Results Section:

[Screenshot: Generated comic panels]

Description: Revealed dynamically after generation, this section displays the comic title and the generated panels in a grid. Each panel card shows the AI-generated illustration with its caption or dialogue below. A download button lets the user save the finished comic. The viewport automatically smooth-scrolls to this section upon generation.

Conclusion:

ComicCraft is a generative AI platform built with FastAPI that streamlines comic creation. It uses Google's Gemini models to instantly generate structured story scripts and matching panel illustrations in Superhero, Manga, and Cartoon styles. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost creativity and productivity for students, writers, and educators, with potential for future scalability. Looking ahead, ComicCraft offers significant opportunities for expansion, such as multi-language stories, character consistency across pages, speech-bubble overlays, and user authentication to save comic history. By continuously refining the underlying prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive visual storytelling assistant.